PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies — Complete Assessment Report

Includes problem statements, objectives, assumptions, algorithms, flowcharts, pseudocode, Python programs, test cases, sample outputs, screenshot headings, and conclusions.

Contents

1. Attendance Recovery Computation
2. ATM Transaction Sequence Simulation
3. Multi-Product Shopping Budget Constraint
4. Repeated Transaction Detection
5. Password Policy Validation Engine
6. Student Feedback Sentiment Scoring
7. Text Compression Using Character Runs
8. Student Marks Matrix Analysis
9. Weekly Electricity Consumption Matrix
10. Food Waste Monitoring System

1. Attendance Recovery Computation

1. Problem statement

Calculate the current attendance percentage and the minimum number of consecutive future classes a student must attend to reach a target percentage.

2. Objectives

- Read classes held, classes attended, and target percentage.
- Validate that total classes are positive, attended classes are within 0..total, and target is between 0 and 100.
- Calculate current percentage.
- If below target, repeatedly add one attended class and one held class until the target is reached.
- Display current percentage and required classes.

3. Assumptions

- total classes is greater than zero.
- Each future class is attended.
- The target percentage is supplied by the teacher; default test uses 75%.

4. Algorithm

11. Read classes held, classes attended, and target percentage.
12. Validate that total classes are positive, attended classes are within 0..total, and target is between 0 and 100.
13. Calculate current percentage.
14. If below target, repeatedly add one attended class and one held class until the target is reached.
15. Display current percentage and required classes.

5. Flowchart

```
flowchart TD
    A([Start]) --> B[Read input values]
    B --> C{Inputs valid?}
    C -- No --> D[Display validation message]
    C -- Yes --> E[Process using conditions / loops / lists]
    E --> F[Display results]
    D --> G([End])
    F --> G
```

6. Pseudocode

```
START
  Read classes held, classes attended, and target percentage.
  Validate that total classes are positive, attended classes are within 0..total, and target is between 0
and 100.
  Calculate current percentage.
  If below target, repeatedly add one attended class and one held class until the target is reached.
  Display current percentage and required classes.
STOP
```

7. Python code

See [`main.py`](main.py). The program uses only Python's standard language features; no third-party packages are needed.

8. Test cases

- Input: 20, 12, 75 -> current 60.00%; required 12 classes.
- Input: 20, 16, 75 -> target already reached.

- Input: 0, 0, 75 -> invalid attendance details.
- Input: 20, 12, 100 -> explains that the target cannot be reached through future attendance alone.

9. Conclusion

This case study applies basic computational thinking and Python programming concepts to solve a practical problem. The program validates the stated inputs, processes them according to the assumptions, and displays the result.

Complete Python Program

```
total = int(input("Total classes held: "))
attended = int(input("Classes attended: "))
target = float(input("Target attendance percentage: "))

if total <= 0 or attended < 0 or attended > total or target <= 0 or target > 100:
    print("Invalid attendance details.")
else:
    current = attended / total * 100
    print(f"Current attendance: {current:.2f}%")
    extra = 0
    while (attended + extra) / (total + extra) * 100 < target:
        extra += 1
    if extra == 0:
        print("Target already reached.")
    else:
        print(f"Consecutive classes to attend: {extra}")
```

Test Cases

```
TEST CASES
==========
Input: 20, 12, 75 -> current 60.00%; required 12 classes.
Input: 20, 16, 75 -> target already reached.
Input: 0, 0, 75 -> invalid attendance details.
Input: 20, 12, 100 -> explains that the target cannot be reached through future attendance alone.

Sample stdin values are in sample_input.txt. The automated audit also checks the exact expected output for the
primary test.
```

Sample Output

```
Current attendance: 60.00%
Consecutive classes to attend: 12
```

Output Screenshot



2. ATM Transaction Sequence Simulation

1. Problem statement

Simulate a sequence of deposits and withdrawals, reject invalid transactions, and report the final balance and transaction counts.

2. Objectives

- Read an opening balance and a sequence of transaction commands.
- For each command, accept deposit or withdraw and an amount.
- Reject negative amounts and withdrawals exceeding the available balance.
- Update balance only for accepted transactions.
- Display final balance and accepted/rejected counts.

3. Assumptions

- This is a classroom simulation and does not connect to a real bank.
- A transaction amount must be greater than zero.
- Commands are entered as D for deposit, W for withdrawal, and X to finish.

4. Algorithm

16. Read an opening balance and a sequence of transaction commands.
17. For each command, accept deposit or withdraw and an amount.
18. Reject negative amounts and withdrawals exceeding the available balance.
19. Update balance only for accepted transactions.
20. Display final balance and accepted/rejected counts.

5. Flowchart

```
flowchart TD
    A([Start]) --> B[Read input values]
    B --> C{Inputs valid?}
    C -- No --> D[Display validation message]
    C -- Yes --> E[Process using conditions / loops / lists]
    E --> F[Display results]
    D --> G([End])
    F --> G
```

6. Pseudocode

```
START
  Read an opening balance and a sequence of transaction commands.
  For each command, accept deposit or withdraw and an amount.
  Reject negative amounts and withdrawals exceeding the available balance.
  Update balance only for accepted transactions.
  Display final balance and accepted/rejected counts.
STOP
```

7. Python code

See [`main.py`](main.py). The program uses only Python's standard language features; no third-party packages are needed.

8. Test cases

- Opening 100; D 50; W 30; X -> final Rs. 120.00, accepted 2, rejected 0.
- Opening 20; W 30; X -> final Rs. 20.00, accepted 0, rejected 1.
- Opening -1 -> invalid opening balance.

10. Conclusion

This case study applies basic computational thinking and Python programming concepts to solve a practical problem. The program validates the stated inputs, processes them according to the assumptions, and displays the result.

Complete Python Program

```
balance = float(input("Opening balance: "))
accepted = 0
rejected = 0
if balance < 0:
    print("Invalid opening balance.")
else:
    while True:
        command = input("Transaction (D/W/X): ").strip().upper()
        if command == "X":
            break
        if command != "D" and command != "W":
            print("Invalid transaction type.")
            rejected += 1
            continue
        amount = float(input("Amount: "))
        if amount <= 0:
            print("Invalid amount.")
            rejected += 1
        elif command == "W" and amount > balance:
            print("Insufficient balance.")
            rejected += 1
        else:
            if command == "D":
                balance += amount
            else:
                balance -= amount
            accepted += 1
            print(f"Balance: Rs. {balance:.2f}")
    print(f"Final balance: Rs. {balance:.2f}")
    print(f"Accepted transactions: {accepted}")
    print(f"Rejected transactions: {rejected}")
```

Test Cases

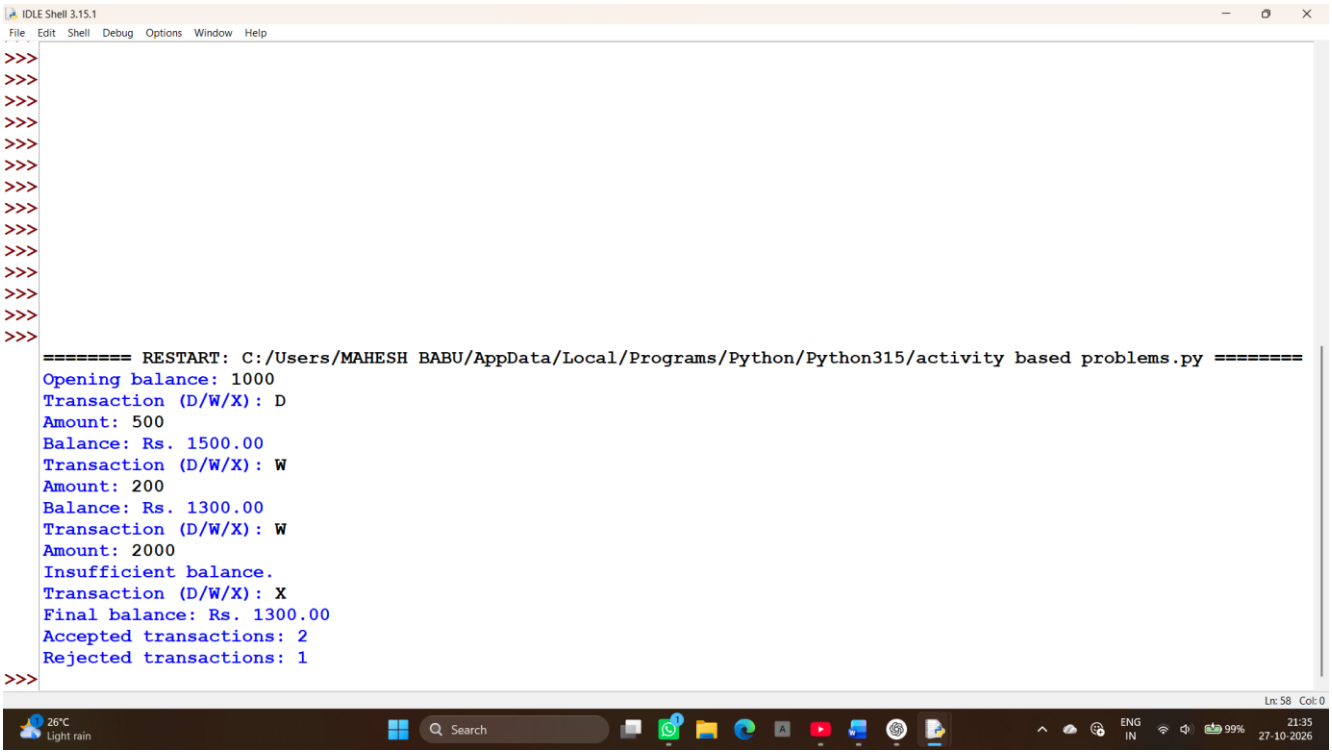
```
TEST CASES
==========
Opening 100; D 50; W 30; X -> final Rs. 120.00, accepted 2, rejected 0.
Opening 20; W 30; X -> final Rs. 20.00, accepted 0, rejected 1.
Opening -1 -> invalid opening balance.

Sample stdin values are in sample_input.txt. The automated audit also checks the exact expected output for the
primary test.
```

Sample Output

```
Balance: Rs. 150.00
Balance: Rs. 120.00
Final balance: Rs. 120.00
Accepted transactions: 2
Rejected transactions: 0
```

Output Screenshot



3. Multi-Product Shopping Budget Constraint

1. Problem statement

Calculate the total cost of a basket of products, apply a threshold discount, and identify whether the basket fits the budget.

2. Objectives

- Read the budget and number of products.
- For each product, read name, unit price, and quantity.
- Reject negative prices, non-positive quantities, or invalid budget/product count.
- Calculate line totals, subtotal, discount and final total.
- Report the amount remaining or the amount over budget.

3. Assumptions

- All prices are in rupees.
- A 10% discount applies when subtotal is at least Rs. 2000.
- This version reports budget status; it does not automatically remove products.

4. Algorithm

21. Read the budget and number of products.
22. For each product, read name, unit price, and quantity.
23. Reject negative prices, non-positive quantities, or invalid budget/product count.
24. Calculate line totals, subtotal, discount and final total.
25. Report the amount remaining or the amount over budget.

5. Flowchart

```
flowchart TD
    A([Start]) --> B[Read input values]
    B --> C{Inputs valid?}
    C -- No --> D[Display validation message]
    C -- Yes --> E[Process using conditions / loops / lists]
    E --> F[Display results]
    D --> G([End])
    F --> G
```

6. Pseudocode

```
START
  Read the budget and number of products.
  For each product, read name, unit price, and quantity.
  Reject negative prices, non-positive quantities, or invalid budget/product count.
  Calculate line totals, subtotal, discount and final total.
  Report the amount remaining or the amount over budget.
STOP
```

7. Python code

See [`main.py`](main.py). The program uses only Python's standard language features; no third-party packages are needed.

8. Test cases

- Budget 2500; one item price 2200 quantity 1 -> subtotal 2200, discount 220, total 1980, within budget.
- Budget 100; one item price 150 quantity 1 -> over budget by Rs. 50.00.
- Count 0 -> invalid budget or product count.

9. Conclusion

This case study applies basic computational thinking and Python programming concepts to solve a practical problem. The program validates the stated inputs, processes them according to the assumptions, and displays the result.

Complete Python Program

```
budget = float(input("Budget (Rs.): "))
count = int(input("Number of products: "))
if budget < 0 or count <= 0:
    print("Invalid budget or product count.")
else:
    subtotal = 0.0
    valid = True
    for i in range(1, count + 1):
        name = input(f"Product {i} name: ").strip()
        price = float(input("Unit price (Rs.): "))
        quantity = int(input("Quantity: "))
        if name == "" or price < 0 or quantity <= 0:
            valid = False
        subtotal += price * quantity
    if not valid:
        print("Invalid product details.")
    else:
        discount = subtotal * 0.10 if subtotal >= 2000 else 0
        total = subtotal - discount
        print(f"Subtotal: Rs. {subtotal:.2f}")
        print(f"Discount: Rs. {discount:.2f}")
        print(f"Final total: Rs. {total:.2f}")
        if total <= budget:
            print(f"Within budget. Remaining: Rs. {budget-total:.2f}")
        else:
            print(f"Over budget by: Rs. {total-budget:.2f}")
```

Test Cases

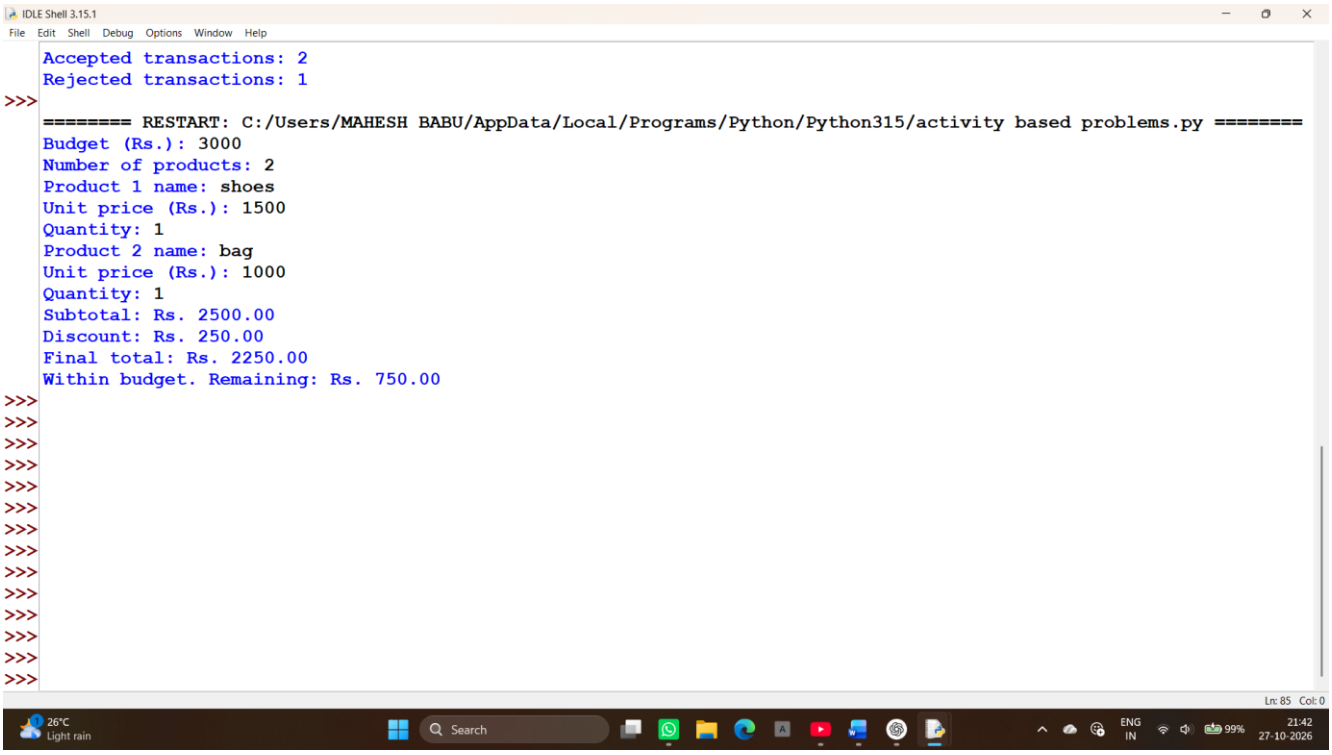
```
TEST CASES
==========
Budget 2500; one item price 2200 quantity 1 -> subtotal 2200, discount 220, total 1980, within budget.
Budget 100; one item price 150 quantity 1 -> over budget by Rs. 50.00.
Count 0 -> invalid budget or product count.

Sample stdin values are in sample_input.txt. The automated audit also checks the exact expected output for the
primary test.
```

Sample Output

```
Subtotal: Rs. 2200.00
Discount: Rs. 220.00
Final total: Rs. 1980.00
Within budget. Remaining: Rs. 520.00
```

Output Screenshot



4. Repeated Transaction Detection

1. Problem statement

Inspect transaction IDs and amounts, identify repeated IDs, and flag transactions whose amount exceeds a supplied limit.

2. Objectives

- Read the number of transactions and alert limit.
- Read each transaction ID and amount into lists.
- Compare each ID with earlier IDs to detect duplicates.
- Flag amounts above the limit.
- Display duplicate IDs and high-value transactions.

3. Assumptions

- Transaction IDs are compared exactly as entered.
- Amounts must be non-negative.
- A duplicate means the same ID appears more than once.

4. Algorithm

26. Read the number of transactions and alert limit.
27. Read each transaction ID and amount into lists.
28. Compare each ID with earlier IDs to detect duplicates.
29. Flag amounts above the limit.
30. Display duplicate IDs and high-value transactions.

5. Flowchart

```
flowchart TD
    A([Start]) --> B[Read input values]
    B --> C{Inputs valid?}
    C -- No --> D[Display validation message]
    C -- Yes --> E[Process using conditions / loops / lists]
    E --> F[Display results]
    D --> G([End])
    F --> G
```

6. Pseudocode

```
START
  Read the number of transactions and alert limit.
  Read each transaction ID and amount into lists.
  Compare each ID with earlier IDs to detect duplicates.
  Flag amounts above the limit.
  Display duplicate IDs and high-value transactions.
STOP
```

7. Python code

See [`main.py`](main.py). The program uses only Python's standard language features; no third-party packages are needed.

8. Test cases

- IDs T1,T2,T1 with amounts 100,700,800 and limit 500 -> duplicate T1; high-value T2 and T1.
- All IDs unique and amounts below limit -> no duplicates and no high-value transactions.
- Negative limit -> invalid count or limit.

9. Conclusion

This case study applies basic computational thinking and Python programming concepts to solve a practical problem. The program validates the stated inputs, processes them according to the assumptions, and displays the result.

Complete Python Program

```
count = int(input("Number of transactions: "))
limit = float(input("High-value limit (Rs.): "))
ids = []
amounts = []
if count <= 0 or limit < 0:
    print("Invalid count or limit.")
else:
    valid = True
    for i in range(count):
        tid = input(f"Transaction {i+1} ID: ").strip()
        amount = float(input("Amount (Rs.): "))
        if tid == "" or amount < 0:
            valid = False
        ids.append(tid)
        amounts.append(amount)
    if not valid:
        print("Invalid transaction details.")
    else:
        duplicates = []
        high = []
        for i in range(count):
            if ids[i] in ids[:i] and ids[i] not in duplicates:
                duplicates.append(ids[i])
            if amounts[i] > limit:
                high.append((ids[i], amounts[i]))
        print("Duplicate IDs:", ", ".join(duplicates) if duplicates else "None")
        print("High-value transactions:")
        if high:
            for tid, amount in high:
                print(f"{tid}: Rs. {amount:.2f}")
        else:
            print("None")
```

Test Cases

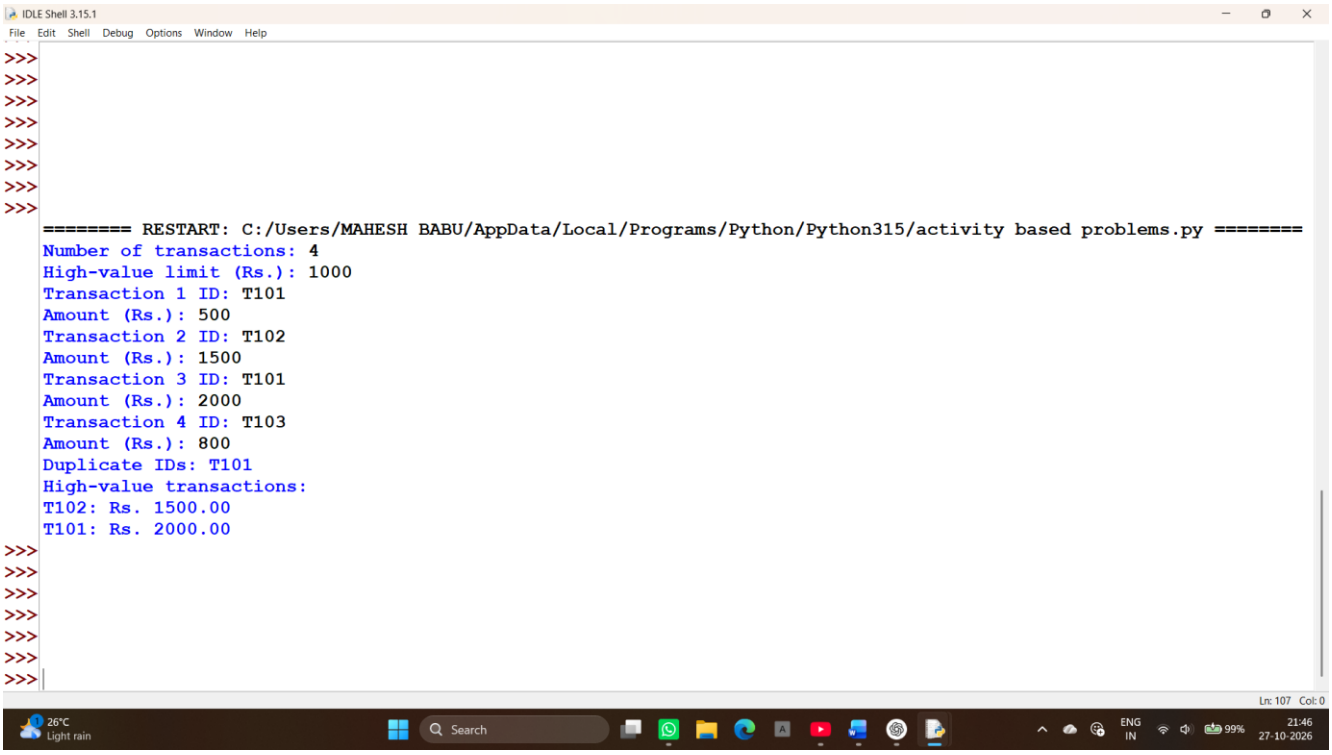
```
TEST CASES
==========
IDs T1,T2,T1 with amounts 100,700,800 and limit 500 -> duplicate T1; high-value T2 and T1.
All IDs unique and amounts below limit -> no duplicates and no high-value transactions.
Negative limit -> invalid count or limit.

Sample stdin values are in sample_input.txt. The automated audit also checks the exact expected output for the
primary test.
```

Sample Output

```
Duplicate IDs: T1
High-value transactions:
T2: Rs. 700.00
T1: Rs. 800.00
```

Output Screenshot



5. Password Policy Validation Engine

1. Problem statement

Check a sample password against a supplied minimum length and character-category rules, and explain which rules fail.

2. Objectives

- Read a password string.
- Check minimum length, at least one uppercase letter, lowercase letter, digit, and special character.
- Collect every failed condition.
- Display valid or invalid and list failed rules.

3. Assumptions

- For this classroom exercise, a special character is any character that is not a letter or digit.
- Do not use this program to store or evaluate real passwords.
- Spaces count as special characters in this simple rule.

4. Algorithm

31. Read a password string.
32. Check minimum length, at least one uppercase letter, lowercase letter, digit, and special character.
33. Collect every failed condition.
34. Display valid or invalid and list failed rules.

5. Flowchart

```
flowchart TD
    A([Start]) --> B[Read input values]
    B --> C{Inputs valid?}
    C -- No --> D[Display validation message]
    C -- Yes --> E[Process using conditions / loops / lists]
    E --> F[Display results]
    D --> G([End])
    F --> G
```

6. Pseudocode

```
START
  Read a password string.
  Check minimum length, at least one uppercase letter, lowercase letter, digit, and special character.
  Collect every failed condition.
  Display valid or invalid and list failed rules.
STOP
```

7. Python code

See [`main.py`](main.py). The program uses only Python's standard language features; no third-party packages are needed.

8. Test cases

- Abcdef1! -> meets sample policy.
- abc -> fails length, uppercase, digit, and special-character rules.
- ABC12345 -> fails lowercase and special-character rules.

.

9. Conclusion

This case study applies basic computational thinking and Python programming concepts to solve a practical problem. The program validates the stated inputs, processes them according to the assumptions, and displays the result.

Complete Python Program

```
password = input("Enter sample password: ")
failed = []
if len(password) < 8:
    failed.append("At least 8 characters")
has_upper = False
has_lower = False
has_digit = False
has_special = False
for ch in password:
    if ch.isupper():
        has_upper = True
    elif ch.islower():
        has_lower = True
    elif ch.isdigit():
        has_digit = True
    else:
        has_special = True
if not has_upper:
    failed.append("At least one uppercase letter")
if not has_lower:
    failed.append("At least one lowercase letter")
if not has_digit:
    failed.append("At least one digit")
if not has_special:
    failed.append("At least one special character")
if len(failed) == 0:
    print("Password meets the sample policy.")
else:
    print("Password does not meet the sample policy.")
    for rule in failed:
        print("-", rule)
```

Test Cases

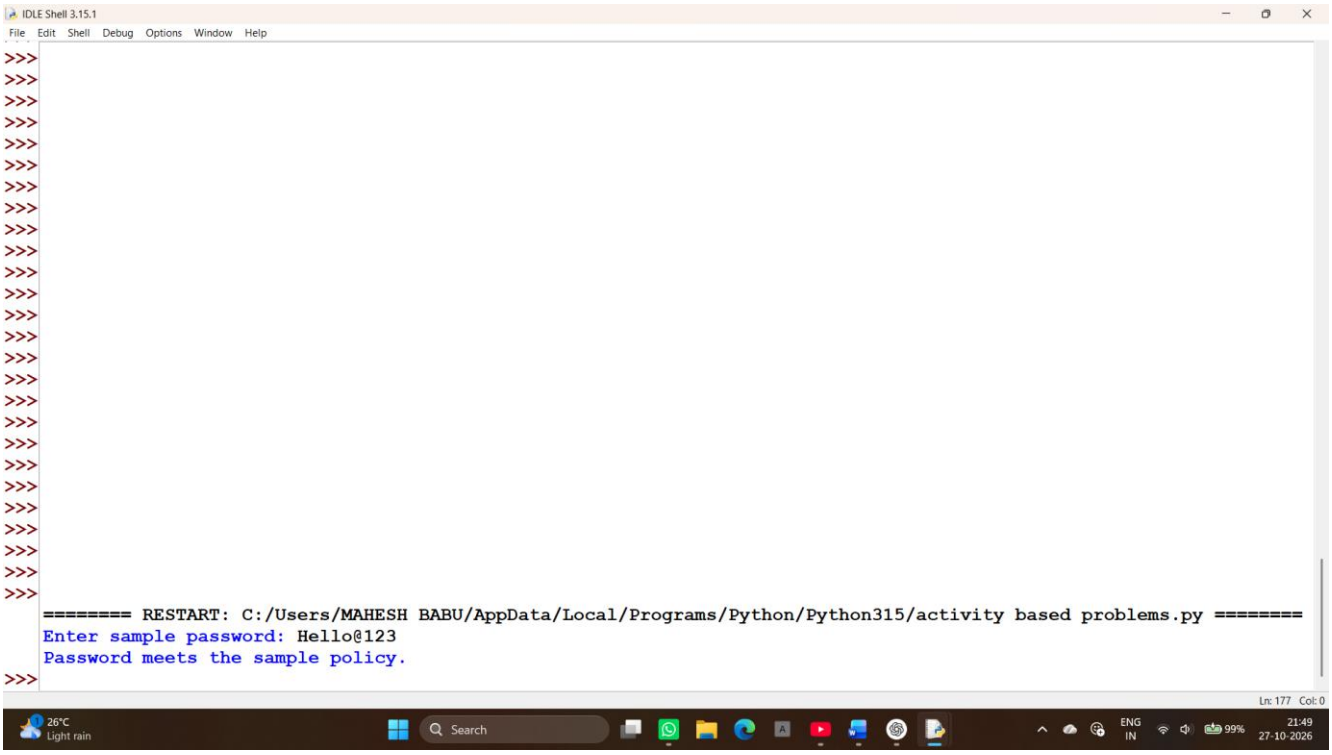
```
TEST CASES
==========
Abcdef1! -> meets sample policy.
abc -> fails length, uppercase, digit, and special-character rules.
ABC12345 -> fails lowercase and special-character rules.

Sample stdin values are in sample_input.txt. The automated audit also checks the exact expected output for the
primary test.
```

Sample Output

```
Password meets the sample policy.
```

Output Screenshot


IDLE Shell 3.15.1
File Edit Shell Debug Options Window Help
>>>
>>>
>>>

6.Student Feedback Sentiment Scoring

1. Problem statement

Score multiple feedback comments using a supplied positive and negative keyword list and classify each comment.

2. Objectives

- Read the number of comments.
- For each comment, convert it to lowercase.
- Count occurrences of words from the positive and negative keyword lists.
- Calculate score as positive count minus negative count.
- Classify positive, negative, or neutral using the score.

3. Assumptions

- Keyword matching is simple substring matching and is not a full language understanding system.
- Positive words: good, helpful, excellent. Negative words: bad, poor, difficult.
- A score above zero is Positive, below zero Negative, and zero Neutral.

4. Algorithm

35. Read the number of comments.
36. For each comment, convert it to lowercase.
37. Count occurrences of words from the positive and negative keyword lists.
38. Calculate score as positive count minus negative count.
39. Classify positive, negative, or neutral using the score.

5. Flowchart

```
flowchart TD
    A([Start]) --> B[Read input values]
    B --> C{Inputs valid?}
    C -- No --> D[Display validation message]
    C -- Yes --> E[Process using conditions / loops / lists]
    E --> F[Display results]
    D --> G([End])
    F --> G
```

6. Pseudocode

```
START
  Read the number of comments.
  For each comment, convert it to lowercase.
  Count occurrences of words from the positive and negative keyword lists.
  Calculate score as positive count minus negative count.
  Classify positive, negative, or neutral using the score.
STOP
```

7. Python code

See [`main.py`](main.py). The program uses only Python's standard language features; no third-party packages are needed.

8. Test cases

- "Good and helpful" -> score 2, Positive.
- "Poor and difficult" -> score -2, Negative.
- "The class starts at ten" -> score 0, Neutral.

.

9. Conclusion

This case study applies basic computational thinking and Python programming concepts to solve a practical problem. The program validates the stated inputs, processes them according to the assumptions, and displays the result.

Complete Python Program

```
positive_words = ["good", "helpful", "excellent"]
negative_words = ["bad", "poor", "difficult"]
count = int(input("Number of feedback comments: "))
if count <= 0:
    print("Invalid number of comments.")
else:
    for i in range(1, count + 1):
        comment = input(f"Comment {i}: ").lower()
        positive = 0
        negative = 0
        for word in positive_words:
            positive += comment.count(word)
        for word in negative_words:
            negative += comment.count(word)
        score = positive - negative
        if score > 0:
            label = "Positive"
        elif score < 0:
            label = "Negative"
        else:
            label = "Neutral"
        print(f"Score: {score}; Sentiment: {label}")
```

Test Cases

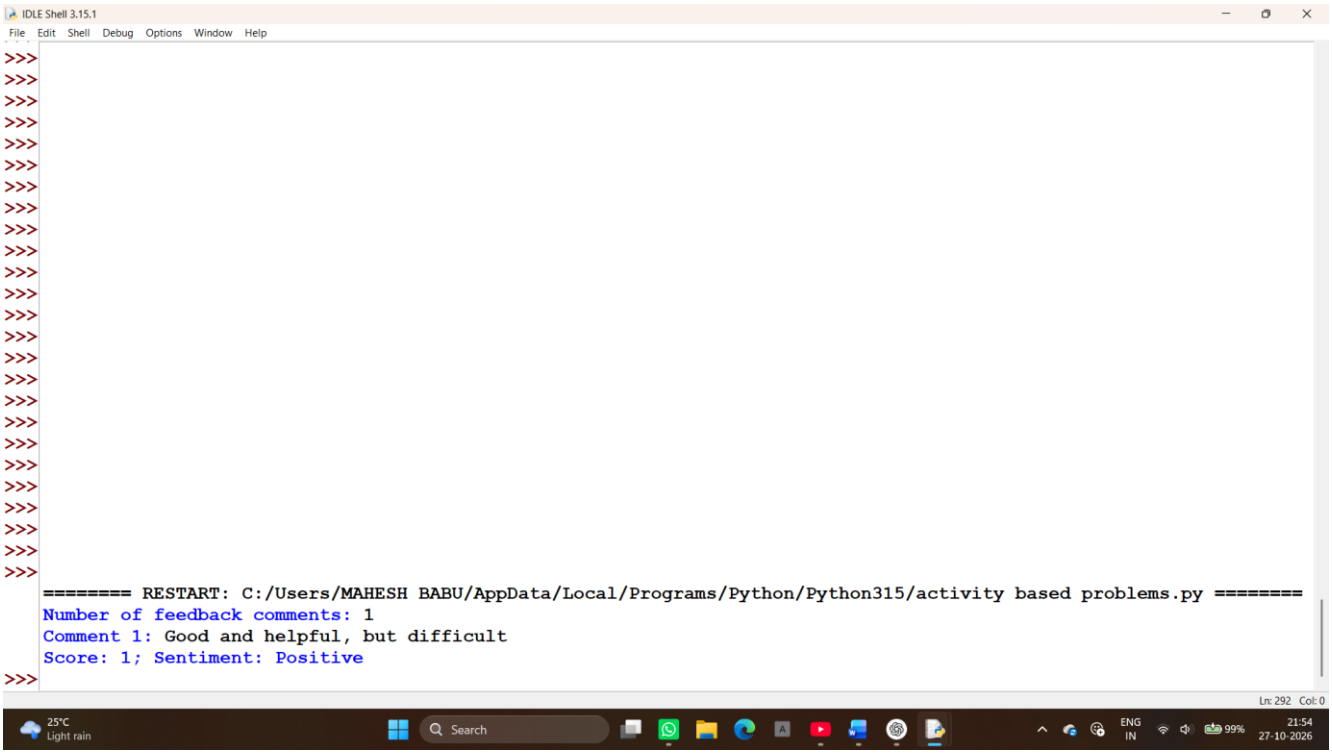
```
TEST CASES
==========
"Good and helpful" -> score 2, Positive.
"Poor and difficult" -> score -2, Negative.
"The class starts at ten" -> score 0, Neutral.

Sample stdin values are in sample_input.txt. The automated audit also checks the exact expected output for the
primary test.
```

Sample Output

```
Score: 2; Sentiment: Positive
Score: -2; Sentiment: Negative
Score: 0; Sentiment: Neutral
```

Output Screenshot


IDLE Shell 3.15.1
File Edit Shell Debug Options Window Help
>>>
>>>
>>>

7. Text Compression Using Character Runs

1. Problem statement

Encode consecutive repeated characters as character-plus-count groups and use the encoded version only when it is shorter.

2. Objectives

- Read a text string.
- If it is empty, report empty input.
- Walk through characters and count consecutive repetitions.
- Build the encoded string using each character followed by its run length.
- Compare lengths and report whether encoding saves space.

3. Assumptions

- The encoding format is character followed by count, for example aaabb becomes a3b2.
- This is an educational run-length encoding example, not a general-purpose compressor.
- The original string is used if the encoding is not shorter.

4. Algorithm

40. Read a text string.
41. If it is empty, report empty input.
42. Walk through characters and count consecutive repetitions.
43. Build the encoded string using each character followed by its run length.
44. Compare lengths and report whether encoding saves space.

5. Flowchart

```
flowchart TD
    A([Start]) --> B[Read input values]
    B --> C{Inputs valid?}
    C -- No --> D[Display validation message]
    C -- Yes --> E[Process using conditions / loops / lists]
    E --> F[Display results]
    D --> G([End])
    F --> G
```

6. Pseudocode

```
START
  Read a text string.
  If it is empty, report empty input.
  Walk through characters and count consecutive repetitions.
  Build the encoded string using each character followed by its run length.
  Compare lengths and report whether encoding saves space.
STOP
```

7. Python code

See [`main.py`](main.py). The program uses only Python's standard language features; no third-party packages are needed.

8. Test cases

- aaabb -> encoded candidate a3b2; encoding saves characters.
- abcd -> candidate a1b1c1d1; original is chosen.
- Empty string -> input is empty.

9. Conclusion

This case study applies basic computational thinking and Python programming concepts to solve a practical problem. The program validates the stated inputs, processes them according to the assumptions, and displays the result.

Complete Python Program

```
text = input("Enter text to encode: ")
if text == "":
    print("Input is empty.")
else:
    encoded = ""
    current = text[0]
    run = 1
    for i in range(1, len(text)):
        if text[i] == current:
            run += 1
        else:
            encoded += current + str(run)
            current = text[i]
            run = 1
    encoded += current + str(run)
    print("Encoded candidate:", encoded)
    if len(encoded) < len(text):
        print("Chosen representation:", encoded)
        print("Encoding saves characters.")
    else:
        print("Chosen representation:", text)
        print("Encoding does not save characters.")
```

Test Cases

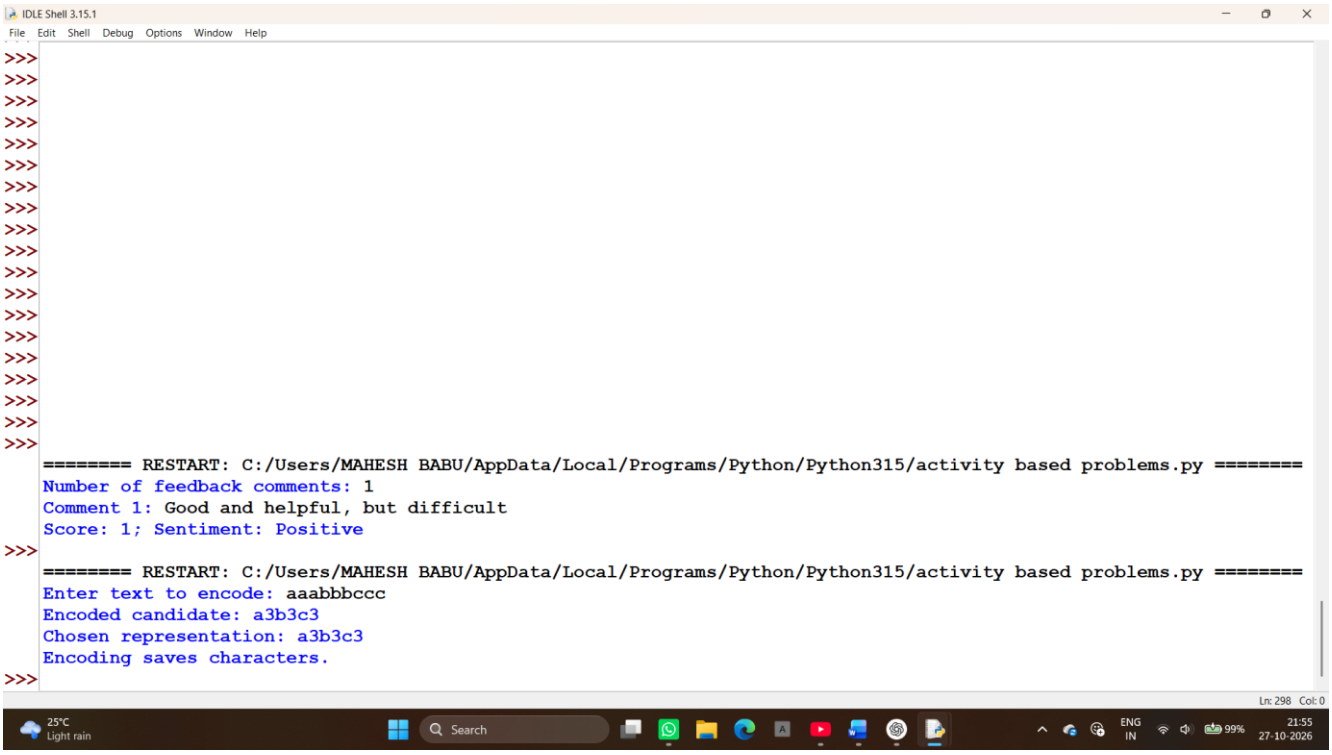
```
TEST CASES
==========
aaabb -> encoded candidate a3b2; encoding saves characters.
abcd -> candidate a1b1c1d1; original is chosen.
Empty string -> input is empty.

Sample stdin values are in sample_input.txt. The automated audit also checks the exact expected output for the
primary test.
```

Sample Output

```
Encoded candidate: a3b2
Chosen representation: a3b2
Encoding saves characters.
```

Output Screenshot



8. Student Marks Matrix Analysis

1. Problem statement

Analyse marks for several students across multiple subjects, calculate each student average and each subject average, and identify the highest student average.

2. Objectives

- Read the number of students and subjects.
- Read student names and marks into a nested list.
- Validate every mark is between 0 and 100.
- Calculate row averages for students and column averages for subjects.
- Display the highest student average.

3. Assumptions

- Each student has a mark for every subject.
- Marks are whole numbers from 0 through 100.
- If multiple students share the highest average, list all of them.

4. Algorithm

45. Read the number of students and subjects.
46. Read student names and marks into a nested list.
47. Validate every mark is between 0 and 100.
48. Calculate row averages for students and column averages for subjects.
49. Display the highest student average.

5. Flowchart

```
flowchart TD
    A([Start]) --> B[Read input values]
    B --> C{Inputs valid?}
    C -- No --> D[Display validation message]
    C -- Yes --> E[Process using conditions / loops / lists]
    E --> F[Display results]
    D --> G([End])
    F --> G
```

6. Pseudocode

```
START
  Read the number of students and subjects.
  Read student names and marks into a nested list.
  Validate every mark is between 0 and 100.
  Calculate row averages for students and column averages for subjects.
  Display the highest student average.
STOP
```

7. Python code

See [`main.py`](main.py). The program uses only Python's standard language features; no third-party packages are needed.

8. Test cases

- 2 students, 2 subjects: A 80,90; B 70,80 -> averages A 85, B 75; subject averages 75 and 85; top A.
- Marks 101 -> invalid student or marks details.
- 0 students -> invalid dimensions.

10. Conclusion

This case study applies basic computational thinking and Python programming concepts to solve a practical problem. The program validates the stated inputs, processes them according to the assumptions, and displays the result.

Complete Python Program

```
students = int(input("Number of students: "))
subjects = int(input("Number of subjects: "))
if students <= 0 or subjects <= 0:
    print("Invalid dimensions.")
else:
    names = []
    matrix = []
    valid = True
    for i in range(students):
        name = input(f"Student {i+1} name: ").strip()
        row = []
        for j in range(subjects):
            mark = int(input(f"{name}, subject {j+1} mark: "))
            if mark < 0 or mark > 100:
                valid = False
            row.append(mark)
        names.append(name)
        matrix.append(row)
    if not valid or "" in names:
        print("Invalid student or marks details.")
    else:
        averages = []
        for row in matrix:
            averages.append(sum(row) / subjects)
        subject_averages = []
        for j in range(subjects):
            total = 0
            for i in range(students):
                total += matrix[i][j]
            subject_averages.append(total / students)
        for i in range(students):
            print(f"{names[i]} average: {averages[i]:.2f}")
        for j in range(subjects):
            print(f"Subject {j+1} average: {subject_averages[j]:.2f}")
        best = max(averages)
        top = []
        for i in range(students):
            if averages[i] == best:
                top.append(names[i])
        print("Top student(s):", ", ".join(top))
```

Test Cases

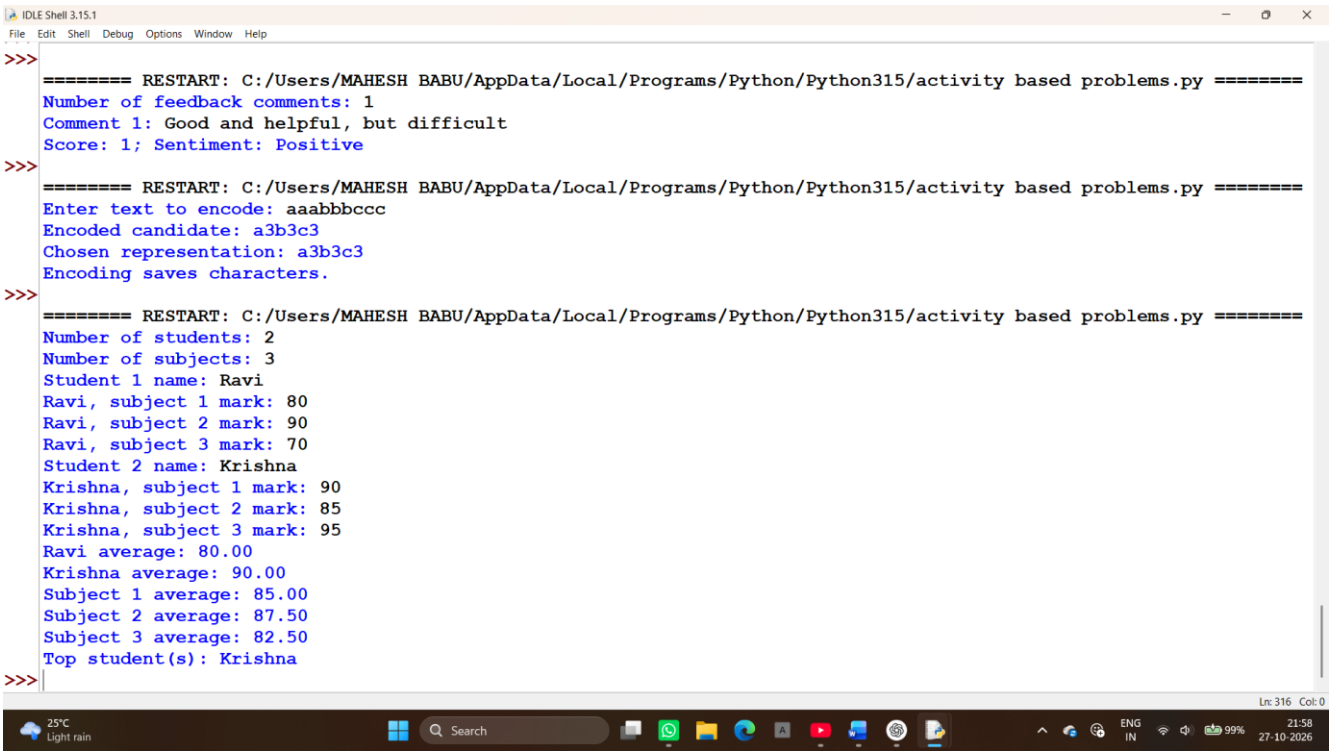
```
TEST CASES
==========
2 students, 2 subjects: A 80,90; B 70,80 -> averages A 85, B 75; subject averages 75 and 85; top A.
Marks 101 -> invalid student or marks details.
0 students -> invalid dimensions.

Sample stdin values are in sample_input.txt. The automated audit also checks the exact expected output for the
primary test.
```

Sample Output

```
A average: 85.00
B average: 75.00
Subject 1 average: 75.00
Subject 2 average: 85.00
Top student(s): A
```

Output Screenshot



9. Weekly Electricity Consumption Matrix

1. Problem statement

Analyse appliance-wise electricity consumption over seven days, calculate daily and appliance totals, and flag daily usage over a supplied limit.

2. Objectives

- Read the number of appliances and the daily alert limit.
- Read each appliance name and seven non-negative daily readings.
- Store readings in a nested list.
- Calculate each day total and each appliance weekly total.
- Flag days whose total exceeds the limit.

3. Assumptions

- Exactly seven daily readings are entered for every appliance.
- Readings are non-negative units of electricity.
- The alert limit is a classroom threshold, not an official utility rule.

4. Algorithm

50. Read the number of appliances and the daily alert limit.
51. Read each appliance name and seven non-negative daily readings.
52. Store readings in a nested list.
53. Calculate each day total and each appliance weekly total.
54. Flag days whose total exceeds the limit.

5. Flowchart

```
flowchart TD
    A([Start]) --> B[Read input values]
    B --> C{Inputs valid?}
    C -- No --> D[Display validation message]
    C -- Yes --> E[Process using conditions / loops / lists]
    E --> F[Display results]
    D --> G([End])
    F --> G
```

6. Pseudocode

```
START
  Read the number of appliances and the daily alert limit.
  Read each appliance name and seven non-negative daily readings.
  Store readings in a nested list.
  Calculate each day total and each appliance weekly total.
  Flag days whose total exceeds the limit.
STOP
```

7. Python code

See [`main.py`](main.py). The program uses only Python's standard language features; no third-party packages are needed.

8. Test cases

- One appliance with readings 1,2,3,4,5,6,7 and limit 5 -> weekly total 28; days 6 and 7 over limit.
- Any negative reading -> invalid appliance or reading details.
- Zero appliances -> invalid count or limit.

9. Conclusion

This case study applies basic computational thinking and Python programming concepts to solve a practical problem. The program validates the stated inputs, processes them according to the assumptions, and displays the result.

Complete Python Program

```
count = int(input("Number of appliances: "))
limit = float(input("Daily alert limit (units): "))
if count <= 0 or limit < 0:
    print("Invalid count or limit.")
else:
    names = []
    readings = []
    valid = True
    for i in range(count):
        name = input(f"Appliance {i+1} name: ").strip()
        row = []
        for day in range(7):
            units = float(input(f"{name}, day {day+1} units: "))
            if units < 0:
                valid = False
            row.append(units)
        names.append(name)
        readings.append(row)
    if not valid or "" in names:
        print("Invalid appliance or reading details.")
    else:
        daily_totals = []
        for day in range(7):
            total = 0
            for i in range(count):
                total += readings[i][day]
            daily_totals.append(total)
            print(f"Day {day+1} total: {total:.2f}")
        for i in range(count):
            print(f"{names[i]} weekly total: {sum(readings[i]):.2f}")
        flagged = []
        for day in range(7):
            if daily_totals[day] > limit:
                flagged.append(str(day+1))
        print("Days over limit:", ", ".join(flagged) if flagged else "None")
```

Test Cases

```
TEST CASES
==========
One appliance with readings 1,2,3,4,5,6,7 and limit 5 -> weekly total 28; days 6 and 7 over limit.
Any negative reading -> invalid appliance or reading details.
Zero appliances -> invalid count or limit.

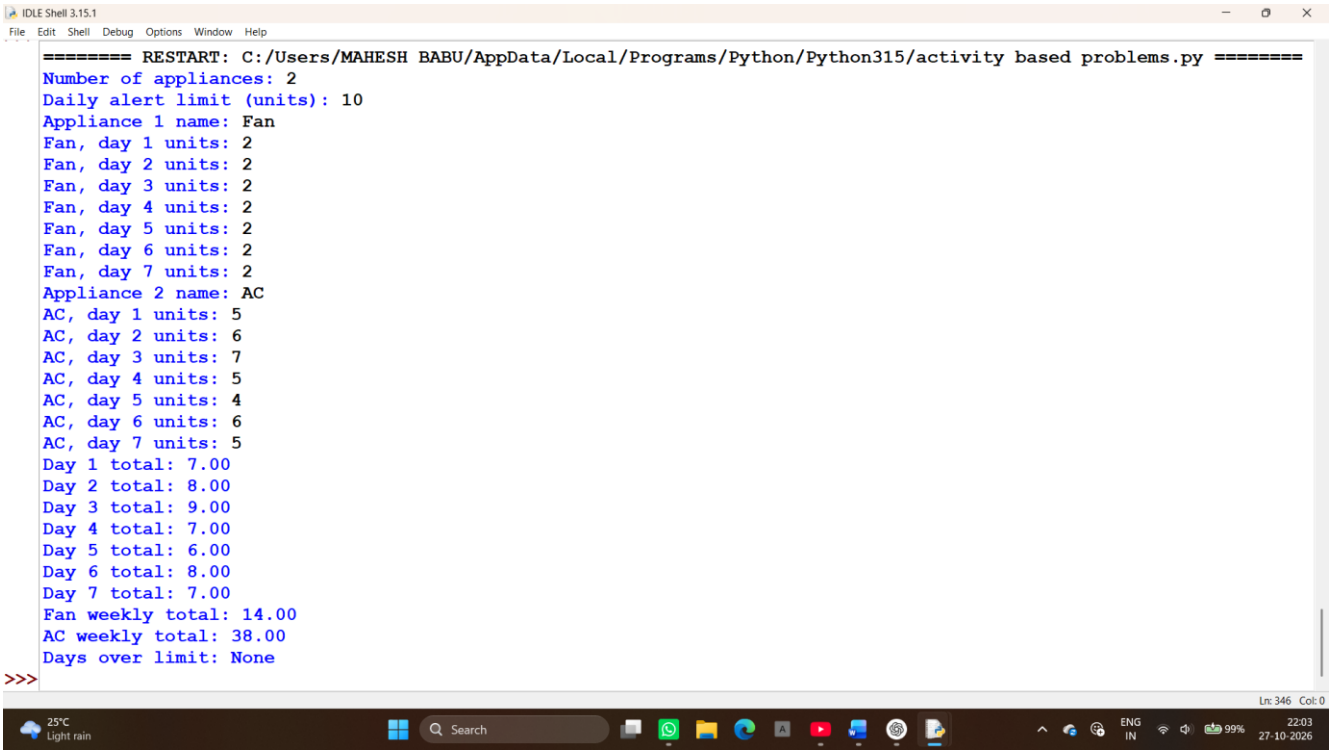
Sample stdin values are in sample_input.txt. The automated audit also checks the exact expected output for the
primary test.
```

Sample Output

```
Day 1 total: 1.00
Day 2 total: 2.00
Day 3 total: 3.00
Day 4 total: 4.00
Day 5 total: 5.00
```

```
Day 6 total: 6.00
Day 7 total: 7.00
Fan weekly total: 28.00
Days over limit: 6, 7
```

Output Screenshot



10. Food Waste Monitoring System

1. Problem statement

Compare meals prepared, served, and left over across multiple days, calculate wastage percentage, and flag days exceeding a supplied threshold.

2. Objectives

- Read the number of days and wastage threshold percentage.
- For each day, read meals prepared and meals served.
- Validate prepared is positive and served is between zero and prepared.
- Calculate leftovers and wastage percentage.
- Report daily figures and flag days over the threshold.

3. Assumptions

- Leftovers equal prepared meals minus served meals.
- Wastage percentage is leftovers divided by meals prepared times 100.
- Threshold is supplied by the user and is between 0 and 100.

4. Algorithm

55. Read the number of days and wastage threshold percentage.
56. For each day, read meals prepared and meals served.
57. Validate prepared is positive and served is between zero and prepared.
58. Calculate leftovers and wastage percentage.
59. Report daily figures and flag days over the threshold.

5. Flowchart

```
flowchart TD
    A([Start]) --> B[Read input values]
    B --> C{Inputs valid?}
    C -- No --> D[Display validation message]
    C -- Yes --> E[Process using conditions / loops / lists]
    E --> F[Display results]
    D --> G([End])
    F --> G
```

6. Pseudocode

```
START
  Read the number of days and wastage threshold percentage.
  For each day, read meals prepared and meals served.
  Validate prepared is positive and served is between zero and prepared.
  Calculate leftovers and wastage percentage.
  Report daily figures and flag days over the threshold.
STOP
```

7. Python code

See [`main.py`](main.py). The program uses only Python's standard language features; no third-party packages are needed.

8. Test cases

- Prepared 100, served 80, threshold 15 -> leftovers 20, wastage 20.00%, day flagged.
- Prepared 100, served 95, threshold 10 -> wastage 5.00%, not flagged.
- Served greater than prepared -> invalid meal counts for that day.

9. Conclusion

This case study applies basic computational thinking and Python programming concepts to solve a practical problem. The program validates the stated inputs, processes them according to the assumptions, and displays the result.

Complete Python Program

```
days = int(input("Number of days: "))
threshold = float(input("Wastage alert threshold (%): "))
if days <= 0 or threshold < 0 or threshold > 100:
    print("Invalid days or threshold.")
else:
    flagged = []
    for day in range(1, days + 1):
        prepared = int(input(f"Day {day} meals prepared: "))
        served = int(input(f"Day {day} meals served: "))
        if prepared <= 0 or served < 0 or served > prepared:
            print("Invalid meal counts for day", day)
        else:
            leftovers = prepared - served
            percentage = leftovers / prepared * 100
            print(f"Day {day}: leftovers={leftovers}, wastage={percentage:.2f}%")
            if percentage > threshold:
                flagged.append(str(day))
    print("Days over wastage threshold:", ", ".join(flagged) if flagged else "None")
```

Test Cases

```
TEST CASES
==========
Prepared 100, served 80, threshold 15 -> leftovers 20, wastage 20.00%, day flagged.
Prepared 100, served 95, threshold 10 -> wastage 5.00%, not flagged.
Served greater than prepared -> invalid meal counts for that day.

Sample stdin values are in sample_input.txt. The automated audit also checks the exact expected output for the
primary test.
```

Sample Output

```
Day 1: leftovers=20, wastage=20.00%
Days over wastage threshold: 1
```

Output Screenshot

